PYTHON FOUNDATIONS - ASSIGNMENT 09

Lists

Collections, Loops & Real-World Data Handling

Scenario-based Python challenges covering list creation, indexing, slicing, mutability, methods, loops, membership, analytics, searching, filtering and nested lists.

indexing | slicing | methods | loops | analytics | nested lists

Name: _____

Date: _____

Batch: _____

BEGINNER TRACK

17 SCENARIO CHALLENGES

QUICK REFERENCE - LIST TOOLKIT

Create, access and slice

`items = ["A", "B", "C"]`
Create a list with square brackets.

`items[0] / items[-1]`
First item / last item.

`items[1:3]`
Start included, stop excluded.

`items[::2] / items[::-1]`
Every second item / reverse view.

`len(items)`
Number of elements in the list.

Change and add

`items[1] = "X"`
Lists are mutable: elements can be changed.

`append(value)`
Add one value at the end.

`insert(index, value)`
Add one value at a chosen position.

`extend(other_list)`
Add every element from another list.

`append(other_list)`
Adds the whole other list as one element.

Remove, search and organize

`remove(value)`
Remove by value.

`pop(index)`
Remove by index and return removed value.

`clear()`
Remove every element.

`value in items`
Membership check.

`count(v) / index(v)`
Count occurrences / find first position.

`sort() / reverse()`
Sort values / reverse current order.

Loop and analyze

`for item in items:`
Loop directly through values.

`for i in range(len(items)):`
Loop through indexes and values.

`total += item`
Accumulator pattern.

`count += 1`
Counter pattern.

`sum / min / max`
Useful built-ins after learning manual logic.

`sentence.split()`
Turns text into a list of words.

CORE IDEA

A list lets one variable hold many values. Unlike strings, lists are mutable, so elements can be added, removed, replaced, sorted and processed with loops.

PART A - LIST BASICS & STRUCTURE

1 Student Roster Inspector

create list | indexing | len()

SCENARIO

A trainer stores five student names in one list and wants a quick summary before starting class.

Your program must:

a. Create a list containing five student names and print the complete list.
b. Print the first, third and last student using indexing.
c. Print the second-last student using negative indexing.
d. Print the total number of students using len().
e. Replace the second student with a new name and print the updated list.

2 Weekly Sales Slice

slicing | step | reverse

SCENARIO

A shop stores sales for seven days in a list. The manager wants different views of the same weekly data.

Your program must:

a. Create a list with seven daily sales values.
b. Print the first three days using slicing.
c. Print everything from index 3 to the end using an omitted stop value.
d. Print every second sales value using step 2.
e. Print the complete list in reverse using slicing.

PART A - LIST BASICS & STRUCTURE

3 Shopping Cart Builder

append() insert() mutability

SCENARIO

An online cart starts with two products. The customer adds products and changes one selection before checkout.

Your program must:

a. Start with a list containing Laptop and Mouse.
b. Append Keyboard to the end of the cart.
c. Insert Headphones at index 1.
d. Replace Mouse with Wireless Mouse using list indexing.
e. Print the final cart and total number of products.

4 Course Catalog Merger

extend() append() comparison

SCENARIO

A training portal has separate frontend and backend course lists that need to be combined.

Your program must:

a. Create frontend = ["HTML", "CSS"] and backend = ["Python", "Java"].
b. Use extend() to add all backend courses into the frontend list.
c. Print the merged list and its length.
d. Create a fresh ["HTML", "CSS"] list and append the entire backend list once.
e. Print both results and write one comment explaining the difference between append() and extend().

PART B - METHODS, MEMBERSHIP & ORGANIZING

5 Inventory Removal Desk

remove() pop() clear()

SCENARIO

A small store updates inventory when items are sold, discontinued or completely reset for stock-taking.

Your program must:

a. Create a list with five product names.
b. Remove one product by value using remove().
c. Remove the item at index 1 using pop(1) and store the removed value.
d. Print the removed value and the remaining inventory.
e. Finally use clear() and print the empty list.

6 Username Registration Check

in / not in append() index()

SCENARIO

A website keeps a list of existing usernames and must prevent duplicate registrations.

Your program must:

a. Create users = ["aman", "rahul", "priya", "neha"].
b. Take a new username from the user and normalize it to lowercase.
c. If it already exists, print "Username already exists".
d. Otherwise append it and print "Username registered".
e. If the username exists after processing, print its index using index().

PART B - METHODS, MEMBERSHIP & ORGANIZING

7 Class Vote Counter

count() membership analysis

SCENARIO

Students vote for one of three activities: Sports, Coding or Music. Votes are stored in a list.

Your program must:

a. Create a list containing at least ten votes with repeated values.
b. Use count() to print the votes received by Sports, Coding and Music.
c. Check whether "Dance" appears in the vote list.
d. Print which option received the most votes using the counts you calculated.
e. Print the total number of votes using len().

8 Leaderboard Organizer

sort() reverse() min/max

SCENARIO

A game stores player scores in a list and needs ascending and descending leaderboard views.

Your program must:

a. Create a list with at least six different scores.
b. Print the highest and lowest scores using max() and min().
c. Sort the list in ascending order and print it.
d. Sort it again in descending order using sort(reverse=True).
e. Use reverse() once and explain in a comment how reverse() differs from sort().

PART C - LOOPS, ANALYTICS & CONTROL FLOW

9 Exam Result Analyzer

loops | sum | average

SCENARIO

A teacher stores class marks in a list and wants a simple result summary without entering each mark separately.

Your program must:

a. Use marks = [76, 35, 89, 42, 28, 91, 67].
b. Loop through the list and calculate the total manually using an accumulator.
c. Calculate the average using the total and len(marks).
d. Count passed students (marks >= 40) and failed students.
e. Print the highest, lowest and all marks greater than the class average.

10 Transaction Cleaner

continue | filtering | counters

SCENARIO

A payment system receives transaction values where negative numbers are invalid and zero means no payment.

Your program must:

a. Use transactions = [1200, -500, 2300, 0, 850, -100, 3000].
b. Loop through the list and skip negative values using continue.
c. Ignore zero values as well.
d. Calculate total valid sales and count valid transactions.
e. Print the largest valid transaction without including invalid values.

PART C - LOOPS, ANALYTICS & CONTROL FLOW

11 Number Lab

conditions | counters | totals

SCENARIO

A data exercise stores mixed positive and negative integers and asks for several classifications in one pass.

Your program must:

a. Use numbers = [12, -5, 7, 20, -8, 15, 0, 18, 25].
b. Count even and odd numbers, excluding zero from the odd count.
c. Count positive and negative numbers.
d. Calculate the sum of numbers divisible by 5.
e. Print only the positive even numbers.

12 Search and Stop

break | range(len()) | indexing

SCENARIO

A warehouse scans product IDs in order and wants to stop scanning as soon as the requested ID is found.

Your program must:

a. Create a list of at least eight numeric product IDs.
b. Take a product ID to search from the user.
c. Use for i in range(len(list)) so you can access both index and value.
d. When the ID is found, print its index and stop immediately using break.
e. If it is not found, print a clear "Product not found" message.

PART D - INPUT, DUPLICATES, STRINGS & NESTING

13 Five-Item Price Collector

user input | append() | analytics

SCENARIO

A billing program asks the cashier to enter five product prices and stores them for later analysis.

Your program must:

a. Start with an empty prices list.
b. Use a for loop to take exactly five prices from the user and append each one.
c. Print the completed list, total and average price.
d. Print the highest and lowest price.
e. Count how many products cost more than the average.

14 Duplicate Customer Cleaner

unique list | not in | loops

SCENARIO

A customer list contains duplicate names because the same person registered more than once.

Your program must:

a. Use names = ["Aman", "Priya", "Aman", "Rahul", "Priya", "Neha"].
b. Create an empty unique_names list.
c. Loop through names and append a name only when it is not already in unique_names.
d. Print the cleaned list and number of unique customers.
e. Do not use set() for this question.

PART D - INPUT, DUPLICATES, STRINGS & NESTING

15 Sentence Word Analyzer

split() | strings + lists | longest word

SCENARIO

A text tool receives one sentence and turns it into a list of words for analysis.

Your program must:

a. Take a sentence from the user and convert it into a word list using split().
b. Print the word list and total number of words.
c. Loop through the list and find the longest word manually.
d. Count how many words have more than 5 characters.
e. Sort the words alphabetically and print the final list.

16 Student Record Matrix

nested lists | indexing | loops

SCENARIO

A teacher stores each student as [name, marks] inside one larger list.

Your program must:

a. Create students = [["Aman", 80], ["Rahul", 65], ["Priya", 92]].
b. Print the first student name and marks using nested indexing.
c. Loop through all records and print each student with their marks.
d. Count how many students scored at least 75.
e. Find and print the student name with the highest marks.

PART E - FINAL COMBINED CHALLENGE

17

Final Shopping Cart Console

final challenge | combined concepts | real-world

SCENARIO

Build a mini console shopping cart that combines list operations with loops, conditions, membership and input.

Your program must:

a. Start with an empty cart and allow the user to add five product names using append().

b. Print the cart with item numbers using range(len(cart)).

c. Ask for one product to search; print whether it exists and its index if found.

d. Ask for one product to remove; remove it only if it exists, otherwise print a safe message.

e. Remove duplicate product names into a new list without using set().

f. Sort the cleaned cart alphabetically, print the final item count, first item and last item.

g. As a stretch goal, keep accepting cart actions in a while loop until the user enters "exit".

TEST IT WITH THESE CASES

Try duplicate names, mixed uppercase/lowercase product names, a product that does not exist, and a cart where only one item remains after removal.

SUBMISSION CHECKLIST

Before you submit, check these seven things:

1. Your list syntax uses square brackets correctly.
2. Indexes and slices do not go outside the available elements.
3. You used append(), insert(), extend(), remove() and pop() for the right purpose.
4. Your loops process every required element and your counters/totals start correctly.
5. You check membership before removing or indexing a value that may not exist.
6. You used break/continue only where the scenario requires them.
7. You tested at least one normal case and one edge/invalid case for each larger scenario.

STRETCH GOAL

Choose any two scenarios and make them fully user-driven instead of using a predefined list. Then add safe handling so your program does not crash when a searched or removed value is missing.

NEXT TOPIC PREVIEW

After Lists, move to Tuples, Sets and Dictionaries. Lists are the foundation for understanding all three.